Prospección: cómo funciona el flujo

Guía para entender el sistema de confirmación de reuniones de venta, qué piezas tiene y qué construimos. Está escrita para alguien que llega nuevo al proyecto.

---

1. El problema que resolvemos

Cuando un prospecto agenda una reunión de venta, muchas veces no llega. Quien vende pierde ese espacio y no puede ofrecérselo a otra persona.

La idea, tomada de un reel de Instagram, es simple: lo que pasa **entre el momento en que agendan y la reunión** decide si la persona llega. Por eso, automáticamente:

1. Apenas agenda, le enviamos un **video de bienvenida** que explica cómo será la reunión.
2. Un día antes le pedimos que **confirme** por WhatsApp.
3. Si no responde, **insistimos** un par de veces.
4. Si sigue sin responder, **liberamos el cupo** para otra persona.
5. Una hora antes, a quien confirmó le enviamos un **recordatorio con el enlace**.

Resultado esperado: menos inasistencias y prospectos que llegan preparados.

---

2. Reglas que decidimos (y por qué)

Decisión	Por qué
Sin base de datos. Todo se guarda en Google Calendar.	Menos infraestructura que mantener. Las reuniones ya viven en el calendario.
Sin inteligencia artificial. El prospecto responde con un número (1, 2 o 3).	Una IA puede interpretar mal un "claro, ahí estaré". Con números, el sistema siempre sabe en qué estado está la reunión.
Secuencia fija. Siempre las mismas 5 etapas; solo cambian los textos y los tiempos.	Es fácil de entender, de probar y de explicar.
Varias personas, cada una independiente.	Cada vendedor tiene su calendario, su WhatsApp y su propia secuencia. Nadie ve los prospectos de otro.
Aislado de MySlotFy. Repositorio propio: `~/orca/prospeccion`.	Es una herramienta interna de ventas, no parte del producto.

---

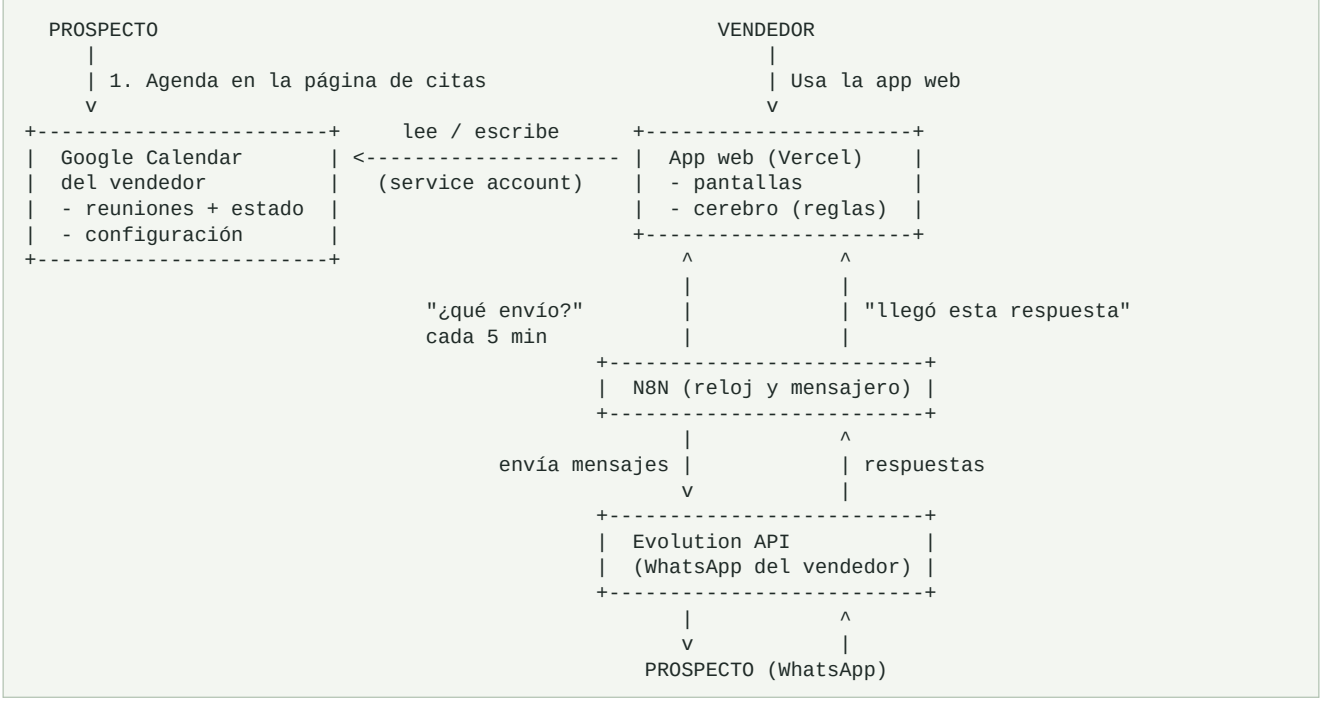
3. Las piezas del sistema

Hay cinco piezas. Conviene entender qué hace cada una antes de ver cómo se conectan.

Pieza	Qué es	Qué hace en este sistema
Google Calendar	El calendario de Google de cada vendedor.	Es "la base de datos". Ahí llegan las reuniones agendadas y ahí guardamos su estado y la configuración del vendedor.
Página de citas (Google Appointment Schedule o Calendly)	Un enlace público donde el prospecto elige un horario.	Crea el evento en el calendario del vendedor, con el teléfono del prospecto en la descripción.

Pieza	Qué es	Qué hace en este sistema
App web (Next.js en Vercel)	La aplicación que construimos.	Tiene la pantalla para el vendedor y el **cerebro**: decide qué mensaje toca enviar y cuándo.
N8N	Una herramienta de automatización, como Zapier, instalada en nuestro servidor.	Es el **reloj** y el **mensajero**: cada 5 minutos le pregunta a la app qué enviar y lo envía. También recibe las respuestas de WhatsApp.
Evolution API	Un servicio que conecta un número de WhatsApp para enviar y recibir mensajes por API.	Envía y recibe los WhatsApp. Cada vendedor conecta su número escaneando un QR.

Diagrama general



Idea clave: la app **decide** y N8N **ejecuta**. Todas las reglas están en un solo lugar (la app), así que solo hay que cambiar una cosa cuando algo cambia.

---

4. La secuencia, paso a paso

Valores por defecto. Cada vendedor los puede cambiar en la pantalla **Secuencia**.

#	Etapa	Cuándo se envía	Solo si...
1	**Bienvenida** (texto + video)	Apenas se detecta la reunión (máximo 5 min)	Siempre
2	**Confirmación** con opciones 1, 2 y 3	24 h antes	Aún no confirmó
3	**Insistencia** (2 veces, cada 3 h)	Después de la confirmación	Aún no confirmó y todavía no es hora de liberar
4	**Liberar cupo**	4 h antes	Aún no confirmó. Se elimina el evento y se avisa al vendedor.
5	**Recordatorio** con el enlace	60 min antes	Ya confirmó

Ejemplo con una reunión el jueves a las 10:00 a. m.

Momento	Qué pasa
Lunes 3:12 p. m.	Ana agenda. Máximo 5 minutos después recibe la **bienvenida con el video**.
Miércoles 10:00 a. m.	Recibe la **confirmación**: "Responde 1, 2 o 3".
Miércoles 1:00 p. m.	No ha respondido, así que recibe la **insistencia 1**.
Miércoles 4:00 p. m.	Sigue sin responder, así que recibe la **insistencia 2**.
Jueves 6:00 a. m.	**Caso A:** nunca respondió. Se **libera el cupo**: se elimina el evento, Ana recibe un mensaje y el vendedor un aviso.
Jueves 9:00 a. m.	**Caso B:** había respondido "1". Recibe el **recordatorio con el enlace**.

Casos especiales

- **Agendó con menos de 4 h de anticipación:** se da por confirmada (acaba de agendar). Solo recibe la bienvenida y el recordatorio.
- **Agendó con menos de 24 h:** la confirmación sale de inmediato, junto con la bienvenida.
- **El vendedor mueve el evento a otra hora en Calendar:** la secuencia se reinicia sola. La bienvenida no se repite.

---

5. Cómo se interpretan las respuestas (sin IA)

El mensaje de confirmación siempre termina así (el sistema lo agrega automáticamente):

```
Responde con un número:
1 - Confirmo
2 - Necesito otro horario
3 - Ya no puedo asistir
```

Antes de comparar, el sistema **limpia** la respuesta: la pasa a minúsculas, le quita tildes, signos y emojis de número. Así, "Sí!", "si" y "1" se leen igual.

El prospecto escribe	Se entiende como	Qué pasa
`1`, `si`, `confirmo`, `confirmado`, `ok`	**Confirmo**	Estado **confirmada**. Recibe un mensaje de agradecimiento. Ya no hay insistencias.
`2`, `reprogramar`, `otro horario`	**Otro horario**	Estado **revisar**. Recibe "en un momento te escribo" y el vendedor recibe un aviso. El vendedor mueve el evento en Calendar y la secuencia se reinicia.
`3`, `cancelar`, `no puedo`	**No puedo**	Se elimina el evento, el prospecto recibe un mensaje y el vendedor un aviso.
Cualquier otra cosa (ej. "claro, ahí estaré")	**No entendido**	**1.ª vez:** se le repiten las opciones. **2.ª vez:** estado **revisar** y aviso al vendedor para que le escriba personalmente.

Otros detalles:

- Lo que el prospecto escriba **antes** de recibir la confirmación (por ejemplo "gracias" tras el video) se ignora.
- Si ya confirmó, solo se tienen en cuenta el 2 y el 3. Un "gracias" no cambia nada.
- Los mensajes de números que no tienen una reunión activa se ignoran.

> **¿Por qué este diseño?** Con IA, un "claro" podría quedar mal clasificado y la reunión quedaría fuera del flujo sin que nadie se enterara. Con números y la regla de las dos oportunidades, **toda** reunión termina confirmada, cancelada, liberada o en manos del vendedor.

---

6. Los estados de una reunión

Cada reunión tiene un estado, que se guarda dentro del propio evento del calendario.

```
                responde 1
   agendada  ------------------>  confirmada
      |  \                            |
      |   \  responde 2 o             | responde 2
      |    \ no entiende 2 veces      v
      |     '------------------->  revisar  (pausa: el vendedor decide)
      |                               |
      | nadie responde                | el vendedor pulsa Confirmar,
      v a tiempo                      | Reactivar o Liberar
   liberada                           v
                                  (vuelve al flujo)

   responde 3 (desde agendada o confirmada)  ->  cancelada
```

Estado	Significado
`agendada`	Esperando a que confirme.
`confirmada`	Confirmó. Solo falta el recordatorio.
`revisar`	**Pausada.** Necesita que el vendedor haga algo. Aparece arriba y resaltada en la app.
`liberada`	No confirmó a tiempo. El evento se eliminó.
`cancelada`	El prospecto canceló. El evento se eliminó.

---

7. Dónde se guarda cada cosa (sin base de datos)

Google Calendar permite guardar datos ocultos dentro de cada evento, en un campo llamado `extendedProperties`. El usuario no los ve, pero la API sí. Ahí guardamos todo.

7.1 En cada reunión

Dato	Ejemplo	Para qué sirve
`pf_estado`	`agendada`	El estado actual.
`pf_telefono`	`573001234567`	A qué número escribir. Se lee de la descripción del evento.
`pf_nombre`	`Ana Pérez`	Para personalizar los mensajes.
`pf_enviados`	`bienvenida,confirmacion`	Qué etapas ya se enviaron. Así **nunca se envía un mensaje dos veces**.
`pf_inicio_ref`	`2026-11-06T10:00…`	La hora original. Si cambia, sabemos que el evento se reprogramó.
`pf_no_entendido`	`1`	Cuántas respuestas no entendidas lleva.
`pf_motivo`	`reprogramar`	Por qué la reunión está en revisión.

> Todos los campos empiezan con `pf_` (de "prospección") para no chocar con otros datos.

7.2 Configuración de cada vendedor

Se guarda en un **evento técnico** dentro del calendario del propio vendedor, con fecha **1 de enero de 2000**, para que nunca lo vea. Contiene sus textos, tiempos, URL del video, su instancia de WhatsApp y su número personal para avisos.

---

8. Cómo funciona por dentro

8.1 El motor (cada 5 minutos)

N8N llama a la app: `POST /api/motor/tick`.

La app revisa todos los calendarios que se compartieron con ella.

Para cada vendedor activo, lee su configuración y sus reuniones de los próximos 14 días.

Para cada reunión, el cerebro (`sequence.ts`) decide:

- Si es nueva, la registra y prepara la bienvenida.
- Si cambió de hora, la reinicia.
- Si ya es hora de una etapa que no se ha enviado, la prepara.

La app guarda el nuevo estado en el evento y le devuelve a N8N la lista de mensajes por enviar.

N8N envía cada mensaje por Evolution API. Si lleva video, lo envía como video con el texto.

Si un envío falla, N8N avisa a la app (`/api/motor/fallo`) y la reunión pasa a **revisar**.

8.2 Las respuestas (cada vez que llega un WhatsApp)

1. El prospecto responde y Evolution API le envía el mensaje a N8N (webhook).
2. N8N se lo pasa a la app: `POST /api/motor/respuesta`.
3. La app identifica al vendedor (por la instancia de WhatsApp) y la reunión (por el teléfono).
4. Interpreta la respuesta (sección 5), guarda el estado y devuelve la respuesta que hay que enviar.
5. N8N la envía.

8.3 Seguridad entre N8N y la app

N8N envía en cada llamada un encabezado `x-motor-token` con una clave secreta (`MOTOR_TOKEN`). Si la clave no coincide, la app responde **401 No autorizado**.

---

9. Lo que construimos

9.1 La app web (`~/orca/prospeccion`)

Tecnologías: **Next.js 16, TypeScript, Tailwind CSS 4 y shadcn/ui**, desplegada en **Vercel**.

Pantalla	Para qué
`/login`	Entrar con la cuenta de Google. Solo pueden entrar los emails autorizados.
`/onboarding`	Primera vez: explica cómo compartir el calendario con el sistema.
`/` **Reuniones**	Próximas reuniones con su estado, qué se envió y qué viene. Botones: **Confirmar**, **Reactivar** y **Liberar cupo**.
`/secuencia`	Editar textos y tiempos de las 5 etapas, con una vista previa tipo burbuja de WhatsApp.
`/ajustes`	Conectar el WhatsApp con QR, la palabra que identifica las reuniones de venta en el título, el WhatsApp personal para avisos y el interruptor para activar o pausar.

9.2 Mapa de archivos importantes

Archivo	Qué contiene
`src/lib/core/sequence.ts`	**El cerebro.** Las reglas de etapas y respuestas. No habla con internet: solo recibe datos y devuelve decisiones.
`src/lib/core/config.ts`	Valores por defecto y textos iniciales.
`src/lib/google/calendar.ts`	Lectura y escritura en Google Calendar.
`src/lib/motor.ts`	Une el cerebro con Calendar: recorre vendedores y reuniones.
`src/lib/evolution.ts`	Crear la instancia de WhatsApp, obtener el QR y consultar si está conectada.
`src/app/api/motor/*`	Los 3 endpoints que llama N8N: `tick`, `respuesta` y `fallo`.
`src/actions/*`	Server Actions: lo que ejecutan los botones de la app.
`src/schemas/*`	Validaciones con Zod.
`n8n/01-motor.json`	Flujo de N8N que corre cada 5 minutos.
`n8n/02-respuestas.json`	Flujo de N8N que recibe las respuestas de WhatsApp.
`tests/secuencia.test.ts`	Pruebas de los escenarios de la secuencia.

9.3 Los flujos de N8N

01 Motor: `Cada 5 minutos` → `Config` → `Calcular etapas` (llama a la app) → `Un envío por item` → `¿Lleva video?` → `Enviar video` o `Enviar texto` → si falla: `Reportar fallo`.

02 Respuestas: `Webhook Evolution` → `Config` → `¿Mensaje entrante?` → `Interpretar respuesta` (llama a la app) → los mismos nodos de envío.

> **¿Por qué la lógica está en la app y no en N8N?** Así hay un solo lugar con reglas, se puede probar con `pnpm test` y los flujos de N8N quedan cortos y fáciles de leer.

---

10. Puesta en marcha (resumen)

El paso a paso detallado está en el `README.md`.

Google Cloud: crear el proyecto, habilitar Calendar API, crear la **service account** (la "cuenta robot" que lee los calendarios) y el **OAuth Client** (para el login).

Vercel: desplegar la app con las variables de `.env.example`.

N8N: crear dos credenciales Header Auth (`x-motor-token` y `apikey`), importar los dos JSON, poner las URLs en el nodo `Config` y activar.

Cada vendedor:

- Crear su página de citas con el **teléfono obligatorio** y un título reconocible (ej. "Demo").
- Entrar a la app y compartir su calendario con la service account.
- En **Ajustes**: conectar su WhatsApp con QR, poner su número personal y activar la secuencia.

---

11. Cómo probarlo

Corre `pnpm test`. Debe decir **"OK: todos los escenarios pasan"**.

En `/secuencia` pon tiempos cortos para no esperar un día entero.

Agenda una reunión de prueba con **tu propio número** y verifica:

- Llegan la bienvenida y la confirmación.

- Responder `1` deja la reunión **confirmada**.
- Responder `2` la deja en **revisar** y te llega el aviso.
- Responder `3` la **cancela** y desaparece del calendario.
- Escribir algo libre dos veces la deja en **revisar**.
- Si no respondes, la reunión se **libera**.

Revisa las ejecuciones en N8N y el estado en la pantalla **Reuniones**.

---

12. Glosario

Término	Significado
Prospecto	Persona interesada que agendó una reunión de venta.
Service account	Una "cuenta robot" de Google que usa la app para leer y escribir en los calendarios compartidos con ella.
OAuth	El mecanismo de "Entrar con Google".
Webhook	Una URL a la que un sistema envía avisos automáticos ("llegó un mensaje").
Endpoint	Una URL de la app que hace una tarea concreta (ej. `/api/motor/tick`).
Instancia (Evolution)	Un número de WhatsApp conectado. Cada vendedor tiene la suya (`prosp-<nombre>`).
Idempotente	Que ejecutarlo varias veces da el mismo resultado. Gracias a `pf_enviados`, el motor puede correr cada 5 minutos sin repetir mensajes.
Server Action	Función de Next.js que corre en el servidor cuando se pulsa un botón.
extendedProperties	Campo oculto de los eventos de Google Calendar donde guardamos nuestros datos.

---

13. Limitaciones conocidas

- Si el teléfono no está en la descripción del evento, la reunión queda en **revisar**.
- Un mensaje se marca como enviado justo antes de enviarlo. Si Evolution falla, no se reintenta solo: la reunión pasa a **revisar** para que el vendedor se entere.
- El webhook de N8N no exige clave. Alguien que conozca la URL y el número de un prospecto podría simular una respuesta. Mejora pendiente: agregar un token al webhook.
- Solo se revisan reuniones de los próximos 14 días.